

Repository Workflow & Versioning Philosophy

Phase 1 — Documentation Pipeline (New Project)

Step	Owner	Action
1	Claude & GPT	Kick off <code>+project name+</code> — same workflow as Phoenix (well-structured repo, no deployment). Plan to draft PRD, Tech Stack, App Flow, UI design.md step by step via AI Studio, then have Antigravity turn the project into a task list (project → layers → feature → individual function). Ask what docs are needed <i>beyond</i> what's already planned, so Antigravity has everything it needs to build a perfect task list — share the feature list, a documentation plan, and the fresher handbook for reference.
2	Claude	Since Claude knows the project well, generate AI Studio prompts for the docs. Flag which existing docs to reuse (e.g. Trajectory's design.md), share the Phoenix repo (github.com/vaibhv19/phoenix) and a sample AI Studio prompt from a prior project for structure.
3	AI Studio	Generate PRD, Tech Stack, App Flow, UI Design using the prompts.
4	Claude	Create a <code>.md</code> feature list from the raw text output.
5	Claude	Generate AI Studio prompts for the architecture set: System Architecture, Folder Structure, DB Schema, API contracts — same reuse pattern as step 2 (reference Phoenix's equivalents).
6	AI Studio	Generate the architecture-level docs from those prompts. (→ <i>v0.1.0 Docs Complete</i>)
7	GPT	Review the full doc set (planning + architecture), ask for a prompt for Copilot to fix flagged inconsistencies.
8	Copilot	Fixes the inconsistencies.
9	GPT	Take the fixed docs, generate a prompt for Antigravity to produce the phase-wise roadmap (reference the previous project's prompt).
10	Claude	Review that prompt, rewrite to remove shortcomings.
11	Antigravity	Generate the roadmap from the prompt; fix any mismatch against personal philosophy. (→ <i>v0.5.0 Implementation Roadmap Complete</i>)

Repeat for every planned project.

Phase 2 — Implementation Loop (Per Project)

Step	Owner	Action
1	ChatGPT ↔ Antigravity	Share <code>phase_XX.md</code> , get an implementation prompt with proper git workflow, paste into Antigravity (covers code building, testing, first-draft documents).
2	Manual	Set up keys/secrets → UI polish → deploy (if applicable).
3	GPT → Antigravity	GPT generates prompts for Antigravity to bring docs and README up to MAANG level, done phase-wise — same four sub-phases as Phase 3 below (planning docs → engineering docs → learning handbook → README/release docs).
4	Antigravity	Generate prefilled GitHub issues.
5	GitHub	Mark release v1.0.0 . (→ <i>Functional Implementation Complete</i>)

Repeat for every remaining project.

Phase 3 — Retrofit Already-Shipped Projects

Applies to already-completed projects that predate the MAANG-docs standard: CheckMate Analyze, Trajectory, Phoenix. (Conclave is excluded — see note below.)

1. Docs to MAANG level (priority):

- Phase 1: Upgrade planning documents (PRD, Feature List, Tech Stack, App Flow, UI Design, System Architecture)
- Phase 2: Upgrade engineering documents (API Spec, DB Schema, Security, Provider Strategy, WebSocket, Error Handling, Testing)
- Phase 3: Upgrade the Learning handbook into a true internal engineering knowledge base — diagrams, trade-offs, debugging guides
- Phase 4: Upgrade README, backend/frontend READMEs, and release documentation into one cohesive story

→ mark **v1.0.1** (*Documentation Upgrade*)

2. UI polish, if needed → mark **v1.1.0** (*UI/UX Polish*)

Note: v1.0.1 and v1.1.0 are N/A for Conclave — the MAANG-docs idea originated during that build and its UI was already polished, so it never needed retrofitting. Same N/A applies to every upcoming project (Foundry, StudyLink, Synod, Vigil, Aegis, Pulse, Warden), since MAANG-level docs and UI polish are baked into phase-1/phase-2 from the start rather than retrofitted after the fact.

Phase 4 — Profile README

Update the GitHub profile README with improved structure, a sharper one-liner, and refreshed content.

Repository Versioning Philosophy

GitHub Releases document the engineering *journey*, not just the final code. Each version tag marks a real milestone in the project's maturity, so the repo history reads as a story rather than a pile of commits.

Version	Milestone	Includes
v0.1.0	Docs Complete	PRD, Feature List, Tech Stack, App Flow, UI Design, plus System Architecture, Folder Structure, DB Schema, and API contracts — all planning and architecture docs finalized
v0.5.0	Roadmap Complete	Development roadmap generated, features broken into phases, tasks decomposed to individual functions, git workflow and implementation strategy finalized
v1.0.0	Functional Implementation Complete	All planned features implemented, testing completed, initial documentation written, stable working version released
v1.0.1	Documentation Upgrade	Planning docs upgraded, engineering docs completed, README polished, Learning handbook expanded, production-grade documentation
v1.1.0	UI/UX Polish	Visual improvements, accessibility improvements, UX refinements, performance optimizations — final portfolio-quality release

The goal: the repository history tells a complete story — idea → architecture → implementation → refinement — rather than showing only finished code. This demonstrates planning ability, architectural thinking, implementation discipline, and continuous improvement to anyone reviewing the repo cold.

Project Tracker

Project	v0.1.0 Docs Complete	v0.5.0 Roadmap Complete	v1.0.0 Functional Complete	v1.0.1 Docs Upgrade	v1.1.0 UI Polish
CheckMate Analyze	— (pre-dates ladder)	— (pre-dates ladder)	✓	☐	☐
Trajectory	— (pre-dates ladder)	— (pre-dates ladder)	✓	☐	N/A (already polished)
Phoenix	— (pre-dates ladder)	— (pre-dates ladder)	✓	☐	☐
Conclave	— (pre-dates ladder)	— (pre-dates ladder)	✓ (retroactive — built through UI polish, never tagged until now)	N/A	N/A
Foundry	— (predates tag, skipped)	✓	☐	N/A	N/A
StudyLink	✓	☐	☐	N/A	N/A
Synod	☐ (docs generated in AI Studio, not yet added to local folder)	☐	☐	N/A	N/A
Vigil	☐	☐	☐	N/A	N/A
Aegis	☐	☐	☐	N/A	N/A
Pulse	☐	☐	☐	N/A	N/A
Warden	☐	☐	☐	N/A	N/A

Only CheckMate Analyze, Trajectory, and Phoenix are eligible for v1.0.1/v1.1.0 — they predate the MAANG-docs standard and need retrofitting. Everything else either already meets that bar (Conclave) or will meet it during its normal phase-1/phase-2 build (no separate retrofit tag needed).

Tick a box the moment the matching GitHub release tag actually lands — this is meant to track real repo state, not planned state.